
EE5332 Mini-Projects

Overview

Each project follows the same workflow:

1. Write a baseline C implementation (float, no pragmas)
2. Convert to fixed-point (ap_`fixed`), verify against float reference
3. Add HLS pragmas, explore at least 3 design points on the Pareto curve
4. Wrap with AXI-Stream interface, synthesize, generate bitstream
5. Deploy on Pynq, measure performance with DMA + AXI Timer
6. Compare against pure-software (ARM) execution

All projects can use either the FFT Demo harness or the lab5 (FIR filter) DMA streaming harness as a starting point. If you are not using AXI stream for some reason, that needs to be discussed and you can propose your alternatives. Students may also propose their own project with instructor approval, provided it follows the same interface and evaluation structure.

Maximum group size is 3 students. However, for groups, the expectation is that each person in the group has a clear explanation of their contribution to the project.

Multiple students/groups are allowed to do the same project. You may also discuss with each other for ideas - all this needs to be clearly documented in your final report. You should be able to explain your work and ideas clearly.

Interface Convention

- **Data input/output:** AXI-Stream (32-bit packets via DMA)
- **Parameters/coefficients:** AXI-Lite registers (loaded once before streaming)
- **Timing:** AXI Timer IP for cycle-accurate measurement
- **Pynq side:** Python script that allocates DMA buffers, streams data, reads results, and verifies against a numpy/scipy reference

Deliverables

- HLS source code with at least 3 design-point variants
- Synthesis reports for each variant (screenshot or table of LUT/FF/DSP/BRAM/latency/II)
- Vivado block design TCL script
- Pynq Python script with correctness check and timing measurement

-
- Short report (2-3 pages): design choices, Pareto tradeoffs, measured results
-

Project 1: Matrix-Vector Multiply

Problem: Compute $y = A * x$, where A is a matrix (up to 64x64) whose coefficients are loaded once via AXI-Lite, and x is streamed in via AXI-Stream. The result y is streamed out.

Baseline C: Double nested loop – straightforward dot product per output element.

Optimization exploration: - Pipeline the inner (dot product) loop to $II=1$ - Partition rows of A to allow parallel MAC operations - Explore tiling: process multiple output elements concurrently - Trade off: number of DSP48s used vs throughput (vectors/second)

Pynq evaluation: - Python generates random A and x , computes $y_{ref} = A @ x$ with numpy - Stream x through hardware, compare y_{hw} with y_{ref} - Measure: latency per vector, throughput, compare with numpy on ARM - Vary matrix size (16x16, 32x32, 64x64) and report scaling

Why this is a good project: Minimal algorithmic complexity, clean data access pattern, directly exercises array partitioning and loop pipelining. The main challenge is managing BRAM usage for the matrix storage and resolving port conflicts for parallel access.

Project 2: Bitonic Sorting Network

Problem: Sort a fixed-length array of N integers ($N=64, 128, \text{ or } 256$) streamed in via AXI-Stream. Stream sorted output back.

Baseline C: Any standard sort (e.g., merge sort) operating on a buffer of N elements.

HLS-friendly approach: Bitonic sort – a fixed compare-and-swap network with $O(N * \log^2(N))$ comparators in a deterministic pattern. No data-dependent branches, making it ideal for pipelining.

Optimization exploration: - Pipeline compare-and-swap stages - Unroll inner comparator loops to exploit parallelism - Array partitioning to allow simultaneous reads/writes to different elements - Trade off: number of parallel comparators (LUTs) vs latency

Pynq evaluation: - Python generates random integer arrays, streams to hardware - Read back sorted output, verify with `numpy.sort` - Measure: cycles per sort for different array sizes - Compare with Python sort on ARM

Why this is a good project: Introduces sorting networks (a classic parallel algorithm), exercises LUT-heavy designs (comparators, not DSPs), and has a very clean correctness criterion. The deterministic structure maps naturally to hardware.

Project 3: 2D Convolution (Image Filter)

Problem: Apply a $K \times K$ convolution kernel ($K=3$ or 5 , loaded via AXI-Lite) to a grayscale image streamed row-by-row via AXI-Stream. Stream filtered image back.

Baseline C: Triple nested loop over rows, columns, and kernel elements.

HLS-friendly approach: Line buffer + sliding window pattern. Store $K-1$ rows in BRAM, maintain a $K \times K$ window register that slides across each row. Each new pixel shifts the window and produces one output pixel.

Optimization exploration: - Pipeline the pixel processing loop to $II=1$ - Explore kernel size (3×3 vs 5×5) and its resource impact - Partition the line buffer for parallel row access - Trade off: kernel size and image width vs BRAM usage

Pynq evaluation: - Python loads a test image (or generates a synthetic pattern) - Stream pixels row-by-row, read back filtered image - Compare with `scipy.ndimage.convolve` or `numpy.convolve` - Measure: pixels/cycle, total time for a 64×64 or 128×128 image - Compare with OpenCV/numpy on ARM

Why this is a good project: Directly extends the 1D FIR from lab5 to 2D, introduces the line buffer pattern (fundamental to image/video processing), and connects to CNN convolution layers. Students see how a conceptually simple extension (1D to 2D) introduces significant memory management challenges.

Extension Project: Variable-Length FFT

Problem: Implement an FFT that supports runtime-configurable transform lengths (powers of 2 from 16 to 1024). The length N is set via AXI-Lite before each invocation; N complex samples are streamed in and N complex results are streamed out.

Why this is harder than the fixed-size class demo: - Variable loop bounds prevent full unrolling - HLS must generate control logic for runtime iteration counts - Twiddle factor table must cover the

largest N, indexed dynamically - Memory must be sized for worst case (1024 complex samples) - Array partitioning decisions depend on the access pattern across all supported sizes

Starting point: The in-class 32-point FFT demo provides the fixed-size baseline. Students extend it with runtime N selection.

This is recommended only for students comfortable with the HLS tool flow and looking for a deeper challenge.

Indicative Timelines

By	Milestone
Apr 10	Project selection, baseline C implementation
Apr 17	Fixed-point conversion, first HLS synthesis
Apr 24	Optimization exploration (3 design points), Pynq deployment
May 1	Performance measurement, report submission

Projects are expected to be complete before the end-semester begins. The amount of work expected of the project is such that you should be able to finish it within that time, so if you find that it is taking too long, then you need to recalibrate if you are doing the right work.

If you need extensions beyond that time, then presumably it will be graded after the end-semester - you need to let me know as early as possible. Note that in general this is strongly discouraged and will result in stricter evaluation, so it will not really help you to get higher marks.

There will be a viva (about 10 minutes per person) - since this does not require separate preparation and will be based on the report you submit, it will be conducted between May 1-15, and should not interfere with your other exams.